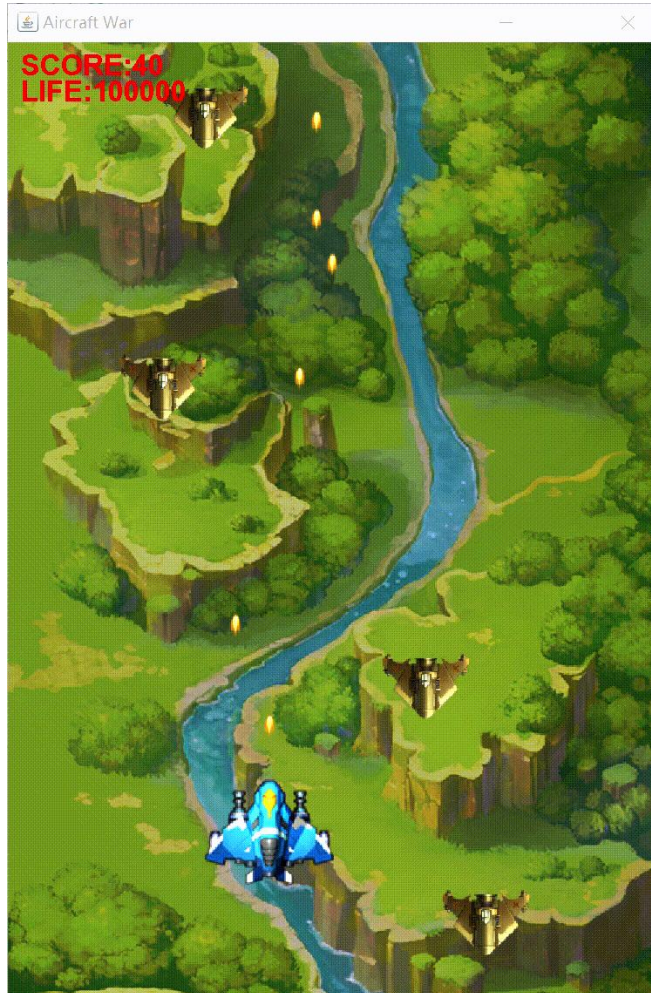
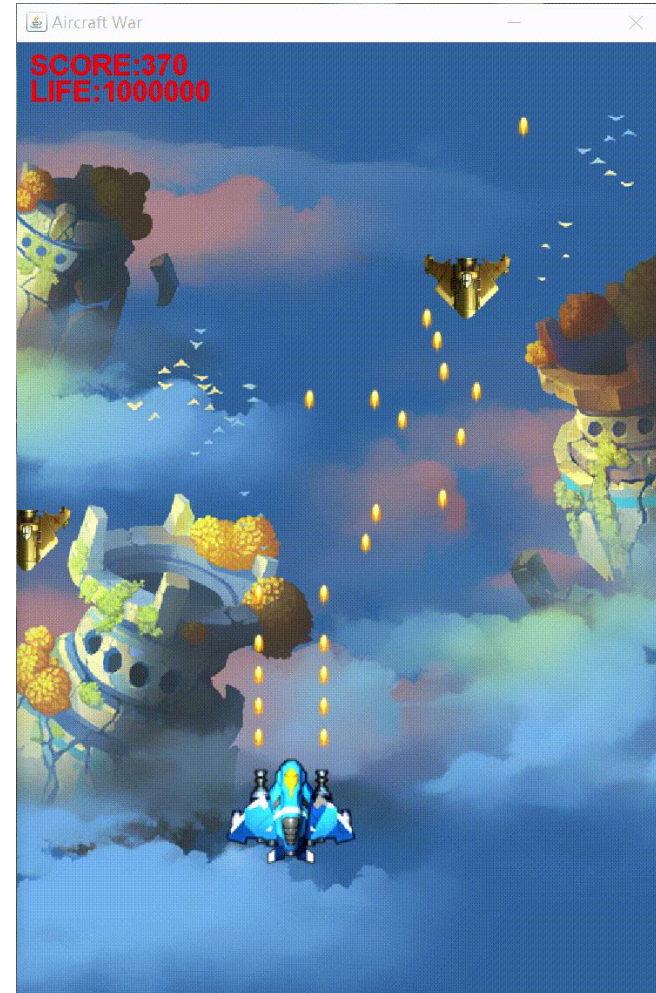
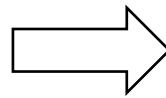


Overall Lab Schedule



Our **Beginning**



Our **Goal**

Overall Lab Schedule

Experiments	Content	Week
 1	Aircraft War System Analysis and Singleton Pattern	Week 11 Monday: 18:45-22:05 PM Room T2510 
2	Factory Pattern	Week 12 Thursday: 14:00-17:20 PM Room T2510 
3	Strategy Pattern	Week 14 Tuesday: 14:00-17:20 PM Room T2510 
4	Audio Control and Multithreading	Week 16 Monday: 14:00-17:20 PM Room T2510 



Experiment 1: Aircraft War System Analysis and Singleton Pattern

Objective

- Understand the core concepts of object-oriented analysis and design
- Learn object-oriented analysis and design methods through practical examples
- Master the skills of drawing UML class diagrams
- Understand the motivation, structure, and implementation of the Singleton Pattern



Experiment Tasks

1. System Analysis

Analyze the "Aircraft War" game system based on functional requirements and design **UML class diagrams**

- Extend the UML Class Diagram of Inheritance Relationships
- Add the EliteEnemy class and Power-up Supply classes to the codebase

2. Design Pattern Application

Identify scenarios suitable for the **Singleton Pattern** and implement it for the Hero Aircraft

- Draw the singleton pattern UML class diagram for the hero aircraft
- Refactor the code and create the hero aircraft using the singleton pattern

Experiment Steps: System Requirements(1)

Character Definitions

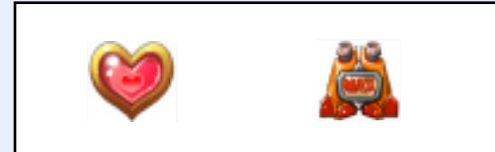
- Hero Aircraft



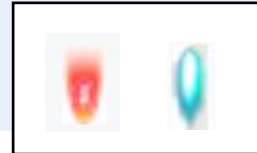
- Enemy Aircraft



- Power-up Supply



- Bullet



Experiment Steps: System Requirements(2)



Game Rules

- Bullet Firing & Trajectory Change
- Power-up Supply Generation & Usage
- Health & Score Calculation
- Enable & disable sound effects

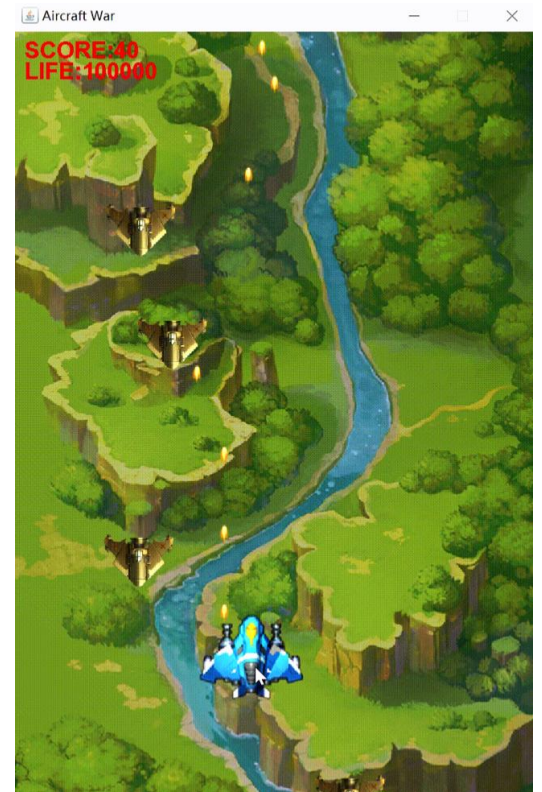
Please refer to the experimental guidebook for detailed requirements!

Experiment Steps: Template Code Import (1)

➤ Template codebase : **AircraftWar-base**

The following core functionalities have already been implemented:

- Game main interface
- Movement of the Hero aircraft and Mob Enemy aircraft
- Bullet firing from the Hero aircraft
- Collision detection
- Score tracking and display, along with the Hero's health points (HP)

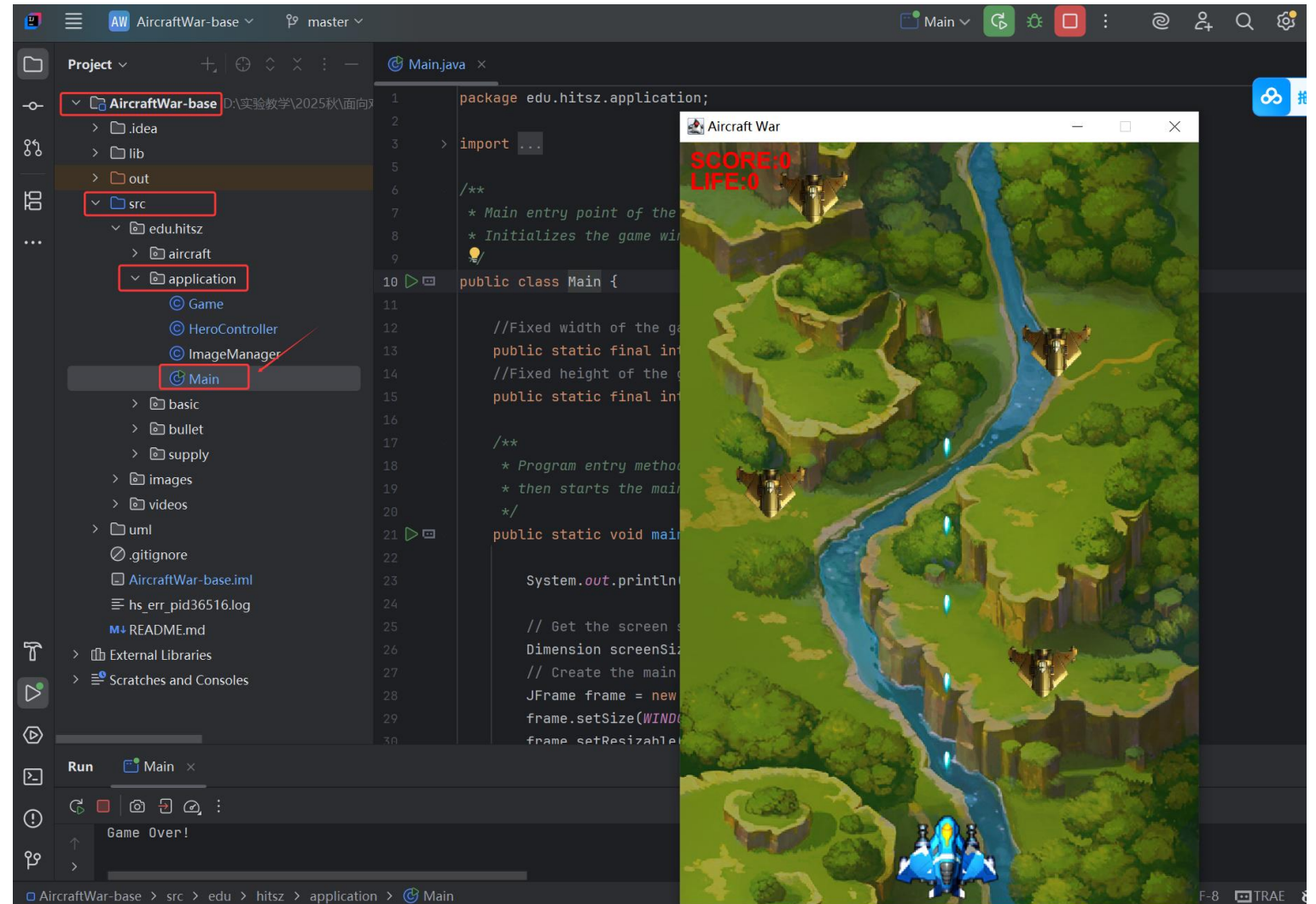


Experiment Steps: Template Code Import (2)

- Download and Install

IntelliJ IDEA Community

- Open the project and
run Main.java

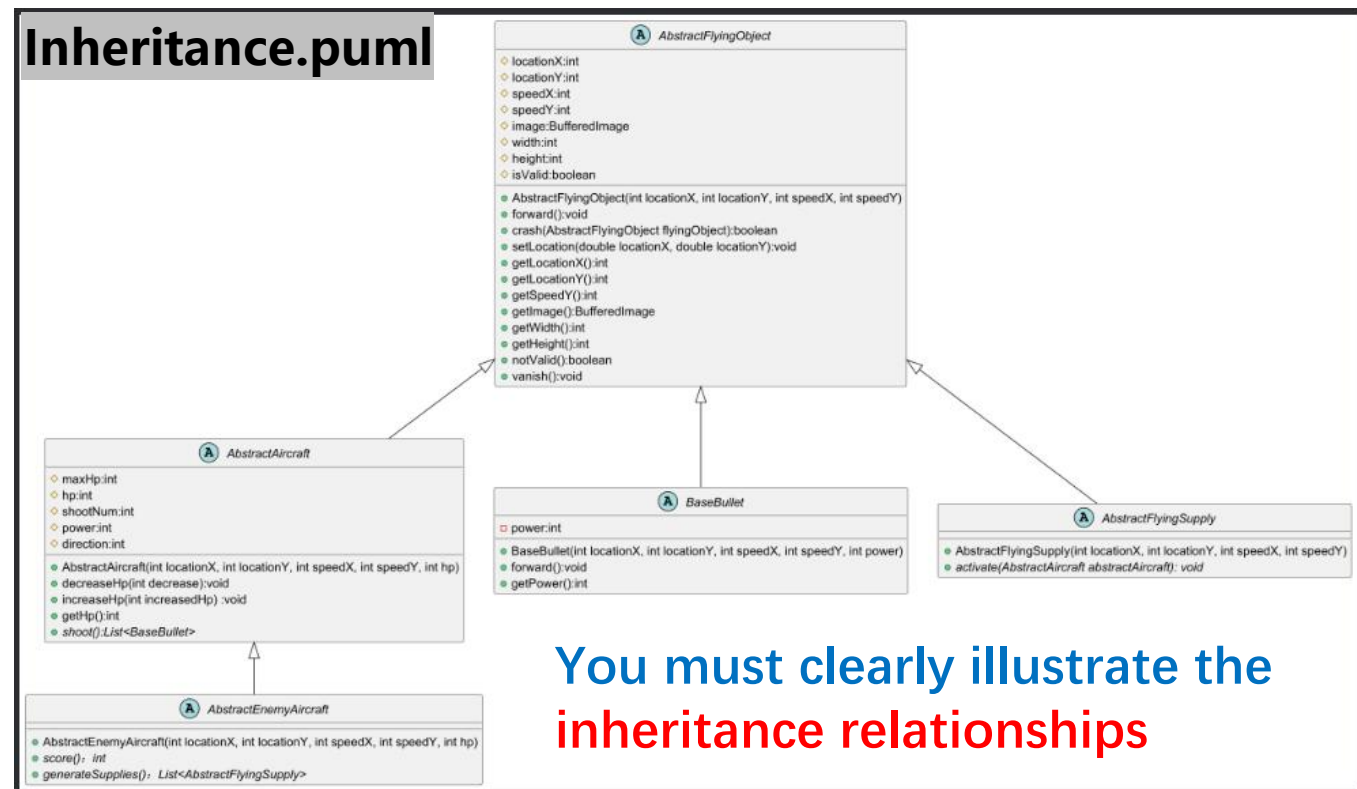


Experiment Steps: Class Diagram Design (1)

➤ **Tasks to be completed:** Based on object-oriented design principles, use the PlantUML plugin to **extend the existing UML class diagram** (uml/Inheritance.puml) by adding the following game entity classes:

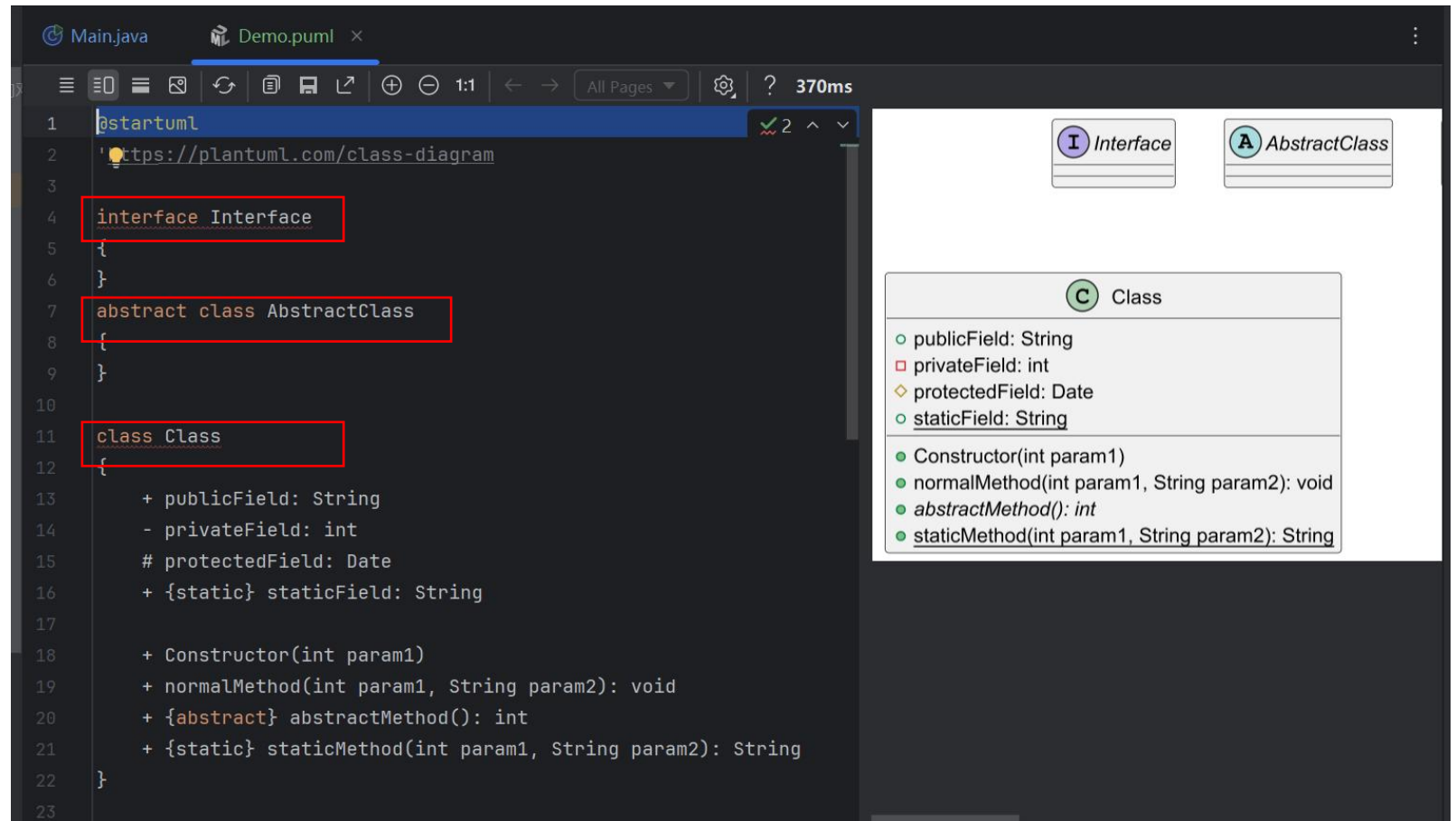
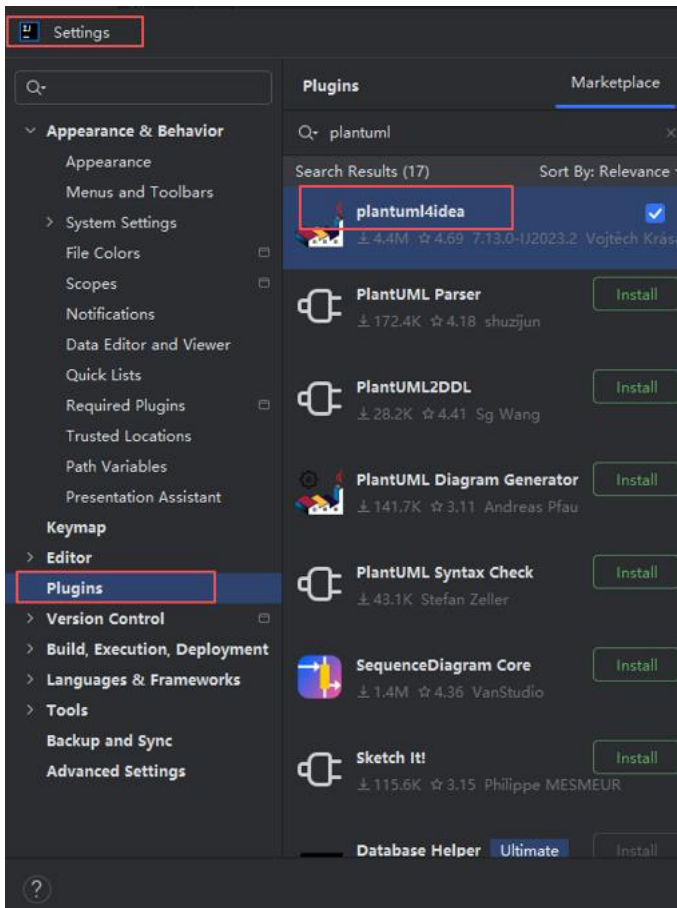


Boss enemy aircraft optional in Exp1



Experiment Steps: Class Diagram Design (2)

PlantUML is a plugin that supports the quick drawing of UML class:



Experiment Steps: Class Diagram Design (3)

- Official documentation: <https://plantuml.com/class-diagram>
- Six types of relationships between classes:

```
abstract class ParentClassA
class SubClassB
ParentClassA <|-- SubClassB

interface InterfaceC
class ImplementationClassD
InterfaceC <|.. ImplementationClassD

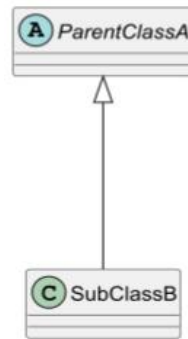
class DependentClassE
class DependencyClassF
DependentClassE ..> DependencyClassF
```

```
class GeneralAssociationClassG
class AssociatedClassH
GeneralAssociationClassG --> AssociatedClassH

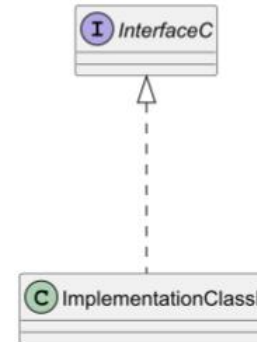
class WholeClassI
class IndependentPartClassJ
WholeClassI o--> IndependentPartClassJ

class WholeClassK
class DependentPartClassL
WholeClassK *--> DependentPartClassL
```

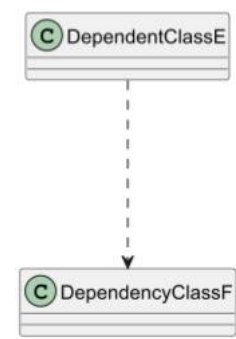
Generalization



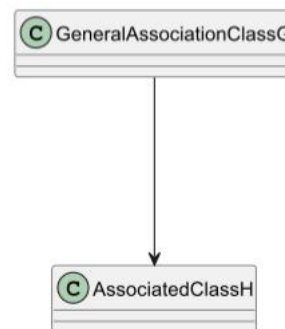
Realization



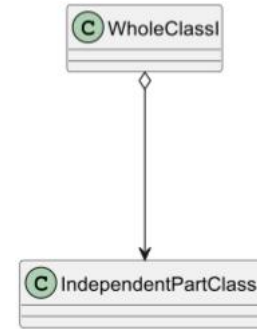
Dependency



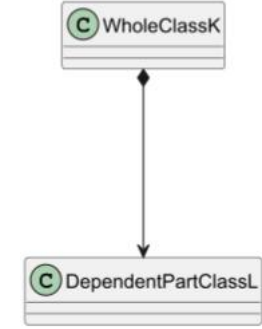
General Association



Aggregation



Composition



Experiment Steps: Code Refactoring (1)

➤ Tasks to be completed:

- Randomly spawn either a mob or **elite enemy aircraft** at regular intervals
- The elite enemy **fires** bullets **straight downward** at fixed intervals
- When destroyed, the elite enemy randomly **drops a power-up supply** (or nothing).



Experiment Steps: Code Refactoring (2)

➤ Tasks to be completed:

- When the hero aircraft collides with a power-up supply, the **supply takes effect automatically**.
- The **health supply** restores a certain amount of health to the hero aircraft, but not exceeding its initial maximum.
- The **firepower supply** requires no concrete implementation in exp1; when activated, it simply prints "FireSupply active!" to the console.



Experiment Steps: Singleton Pattern (1)

➤ Why do we need the singleton pattern?

Scenario Analysis

Requirement:

In the aircraft war game, there is **exactly one hero aircraft per round**, which the player controls to move with the mouse.

Experiment Steps: Singleton Pattern (2)



Let's think about following questions

(1) In which class is the hero aircraft currently created? Does this comply with the **Single Responsibility Principle**?

```
public Game() {  
    heroAircraft = new HeroAircraft(  
        locationX: Main.WINDOW_WIDTH / 2,  
        locationY: Main.WINDOW_HEIGHT - ImageManager.HERO_IMAGE.getHeight() ,  
        speedX: 0, speedY: 0, hp: 1000);  
}
```



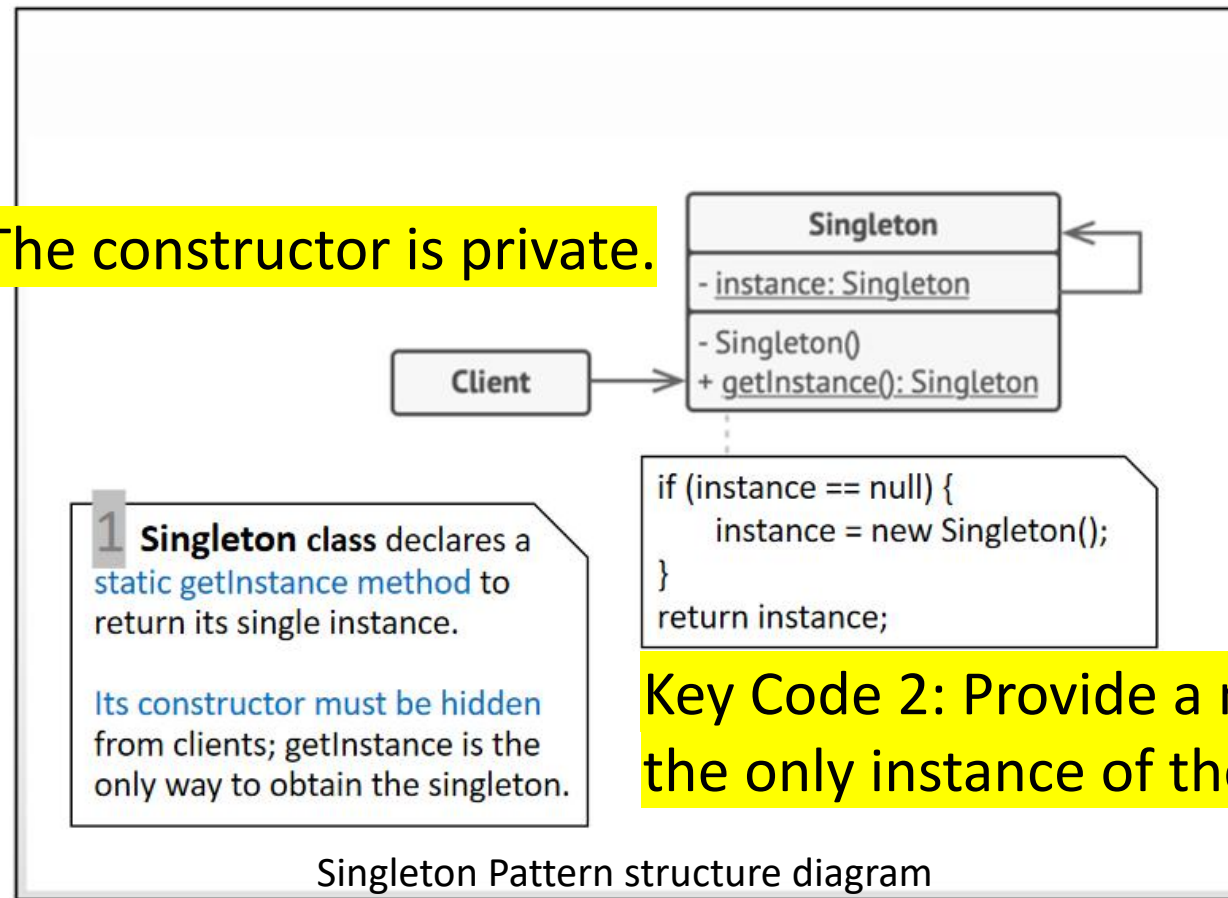
(2) Currently, can we guarantee the **uniqueness** of the hero aircraft instance?

No, external code can freely create a new instance using the new operator.

Experiment Steps: Singleton Pattern (3)

Singleton Pattern is a creational design pattern that ensures a class has only one instance and provides a global point of access to that instance.

Key Code 1: The constructor is private.



1 **Singleton** class declares a static `getInstance` method to return its single instance.

Its constructor must be hidden from clients; `getInstance` is the only way to obtain the singleton.

Key Code 2: Provide a method to access the only instance of the class.

Experiment Steps: Singleton Pattern (4)

➤ **Code examples** (all three are thread-safe) :

① Eager initialization

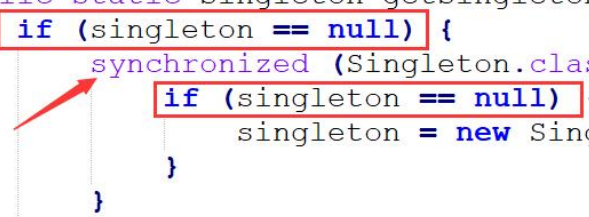
```
public class EagerSingleton {  
    private static EagerSingleton instance = new EagerSingleton ();  
    private EagerSingleton () {}  
    public static EagerSingleton getInstance() {  
        return instance;  
    }  
}
```

② Lazy initialization

```
public class LazySingleton {  
    private static LazySingleton instance = null;  
    private LazySingleton () {}  
    public static synchronized LazySingleton getInstance() {  
        if (instance == null) {  
            instance = new LazySingleton();  
        }  
        return instance;  
    }  
}
```

③ Double-checked locking (DCL)

```
public class Singleton {  
    private volatile static Singleton singleton;  
    private Singleton () {}  
    public static Singleton getInstance() {  
        if (singleton == null) {  
            synchronized (Singleton.class) {  
                if (singleton == null) {  
                    singleton = new Singleton();  
                }  
            }  
        }  
        return singleton;  
    }  
}
```



It is recommended to use this.

Experiment Steps: Singleton Pattern (5)

➤ Tasks to be completed:

- Draw the singleton pattern UML class diagram for the hero aircraft in this game. The client side does not need to be drawn.
- Then Based on your designed UML class diagram, **refactor the code and create the hero aircraft using the singleton pattern.**

Assignment Submission

- **Submission requirements**

- ① Project code (the entire project folder)
- ② Experiment report (use report template)

- **Deadline**

Submit your work to the **HITsz Grader** assignment submission platform within one week.

Login URL: <https://grader.cs-lab.top/>

The default username and password are your student ID.



Let's begin the experiment!

THANK YOU